UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

| | | |
|---|---|---|
| COURSE | : | COMPUTER ORGANIZATION AND ARCHITECTURE |
| COURSE CODE | : | CSC429 |
| EXAMINATION | : | NOVEMBER 2026 (MOCK) |
| TIME | : | 3 HOURS |

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

*This marking scheme consists of 9 printed pages*

© Hak Cipta Universiti Teknologi MARA

# PART A — MARKING GUIDE

## QUESTION A1 (5 marks)

*Topic: Parallel and multiprocessor architecture*

*Source: CSC429 (10).pdf, page(s) 5*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) One characteristic of SMP: all processors share a single common main memory (and typically a common system bus/interconnect), so every processor has equal access to the whole memory and to I/O devices. (Equivalently acceptable: two or more identical processors controlled by one operating system.)

(b) Meaning of "symmetric": the processors are all identical/peer-level and have equal status — no master–slave or supervisor/worker hierarchy. Each processor runs the same copy of the operating system, can execute any task (kernel or user code), and has equal access to shared memory and I/O. The OS treats all processors as a pool of equals, so any processor can be scheduled to run any process, and work is distributed by the OS rather than fixed in hardware.

(c) One advantage of SMP organization: scalability/throughput — additional processors can be added to increase performance (e.g. more concurrent processes handled) with relatively little change to the existing system, and the symmetric design gives good availability/fault tolerance because if one processor fails the remaining processors can continue running the OS and workload. (Either scalability or graceful degradation/availability is acceptable as the single advantage.)

**Unofficial marking guide:**

*• 1 mark: (a) correct characteristic of SMP, e.g. all processors share one common main memory / common bus / single OS controlling identical processors.*

*• 1 mark: (b) states processors are identical/equal in status with no master–slave hierarchy.*

*• 1 mark: (b) explains consequence — same OS on each processor, any processor can run any task, equal access to shared memory/I/O, OS schedules work across the pool.*

*• 1 mark: (c) names a valid advantage, e.g. scalability (easy to add processors for more throughput).*

*• 1 mark: (c) briefly justifies the advantage, e.g. more processors give higher performance/concurrency, or failure of one processor allows the rest to continue (availability/graceful degradation).*

*Model confidence: high.*

*Model note: Question text is clear; no OCR correction needed. Any one valid characteristic/advantage is accepted, so alternative correct answers (e.g. common system bus, single shared OS image, cost-effectiveness) should also earn the marks.*

## QUESTION A2 (5 marks)

*Topic: Data representation and number systems*

Answer: 282810 = 54148

*• 1 mark for identifying the target base.*

*• 1 mark for a correct repeated-division or place-value method.*

*• 1 mark for the correct digit sequence.*

*• 1 mark for the correct base subscript.*

*• 1 mark for a complete, consistent workflow.*

## QUESTION A3 (5 marks)

*Topic: Instruction formats and addressing modes*

*Source: CSC429 (1).pdf, page(s) 5*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective address of the operand, so the operand is fetched directly from memory location A — no further address calculation is performed.

(b) How it is used: The instruction is fetched and its opcode decoded; the CPU takes the address field A as the effective address (EA = A) and places it on the address bus. The memory is read at location A and the operand returned on the data bus is loaded into the operand register/accumulator, where the operation named by the opcode is then executed. Because the address is embedded in the instruction, the operand's location is fixed at assembly/compile time; the programmer or assembler must supply the absolute memory address of the operand in the instruction word.

© Hak Cipta Universiti Teknologi MARA 

(c) Advantage: It is simple and fast — only one memory reference is needed to fetch the operand (no extra address calculation or indirection), and no special address registers are required. Disadvantage: The address field must be large enough to hold a full memory address, which lengthens the instruction; also the address is fixed, so the code is not position-independent and cannot easily access a different location without modifying the instruction (poor flexibility/relocation).

**Unofficial marking guide:**

*• 1 mark: correctly names the technique as direct addressing (accept absolute addressing).*

*• 1 mark: states that the address field A is the effective address of the operand (EA = A).*

*• 1 mark: explains the fetch/execute step — address A is sent to memory, operand read from location A and used by the opcode's operation.*

*• 1 mark: gives ONE valid advantage (e.g. simple/fast, single memory access, no extra address calculation).*

*• 1 mark: gives ONE valid disadvantage (e.g. long address field/instruction, fixed address so not position-independent/relocatable, limited address range).*

*Model confidence: high.*

*Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A, with Memory: Operand) to show the address field pointing directly at the operand; this matches direct addressing. If the figure actually shows an extra indirection arrow (address field pointing to a memory word that holds the operand address), the answer would instead be indirect addressing — the figure should be checked to confirm.*

## QUESTION A4 (5 marks)

*Topic: Computer generations and history*

*Source: CSC429 (21).pdf, page(s) 2*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) The integrated circuit (IC) was invented by Jack Kilby (working at Texas Instruments in 1958). (Robert Noyce of Fairchild also independently developed a silicon IC; either name is acceptable, but Jack Kilby is the standard textbook answer.)

(b) The integrated circuit was first introduced in the third generation of computers (1964–1971), which used SSI and MSI ICs.

(c) Three main components of an integrated circuit are: (1) the transistor (active element), (2) the resistor (passive element), and (3) the capacitor (passive element). (Interconnecting metal tracks and the semiconductor substrate are also part of the IC, but the three fundamental circuit elements are transistor, resistor and capacitor.)

**Unofficial marking guide:**

*• 1 mark: (a) Correctly names Jack Kilby (or Robert Noyce) as inventor of the integrated circuit.*

*• 1 mark: (b) Correctly states third generation (3rd generation) as the computer generation in which the IC was first introduced.*

*• 1 mark: (c) Lists one correct main component of an integrated circuit (e.g., transistor).*

*• 1 mark: (c) Lists a second correct main component (e.g., resistor).*

*• 1 mark: (c) Lists a third correct main component (e.g., capacitor).*

*Model confidence: high.*

*Model note: OCR errors corrected: 'In' → 'In'. For part (a), both Jack Kilby and Robert Noyce are historically credited; the model answer gives Kilby as primary and notes Noyce as acceptable. For part (c), the question asks for 'main components'; the answer uses the three fundamental circuit elements (transistor, resistor, capacitor). If the module's lecture notes list different components (e.g., diodes, inductors, or interconnection), those should be accepted.*

## QUESTION A5 (10 marks)

*Topic: Memory hierarchy and cache*

*Source: CSC429 (4).pdf, page(s) 4*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

Access methods based on hardware implementation of the storage device

1. Sequential access. Data is stored as a linear sequence of records, and to reach a particular record the device must read through all preceding records in order; access time therefore depends on the position of the data and is highly variable. Access is strictly in the order in which the data is physically arranged. *Example:* magnetic tape (also punched paper tape).

2. Direct access. Each block or record has a unique address based on its physical location, so the device can jump close to the required block and then search sequentially within it; access time is still variable but much less order-dependent than pure sequential access. *Example:* magnetic disk / hard disk drive

© Hak Cipta Universiti Teknologi MARA 

(also floppy disk, CD/DVD-ROM).

3. Random access. Any storage location can be selected and accessed directly in essentially the same, constant time, independent of the address or of previous accesses; each location has a unique, electronically addressable identity. *Example:* main memory (RAM / semiconductor memory, e.g. DRAM).

4. Associative access. Data is not addressed by location but retrieved by matching its content (a key or tag) against stored entries in parallel; the hardware compares the search key against all words simultaneously and returns the matching word(s). *Example:* cache memory (associative/fully-associative cache) or a content-addressable memory (CAM) / associative memory.

(If a fourth distinct category is preferred, read-only/ROM-type storage may be cited, but the standard four access methods are sequential, direct, random and associative.)

**Unofficial marking guide:**

- *1 mark: correctly naming/describing sequential access (data read in order, access time depends on position)*
- *1 mark: appropriate example for sequential access (magnetic tape)*
- *1 mark: correctly naming/describing direct access (addressable blocks, direct jump then sequential search)*
- *1 mark: appropriate example for direct access (magnetic disk/HDD)*
- *1 mark: correctly naming/describing random access (any location accessed in constant time, addressable)*
- *1 mark: appropriate example for random access (main memory/RAM)*
- *1 mark: correctly naming/describing associative access (content/key match, parallel comparison)*
- *1 mark: appropriate example for associative access (cache/CAM)*
- *2 marks: overall accuracy and clarity (one mark per two well-explained methods, or 2 marks for four fully correct method+example pairs)*

*Model confidence: high.*

*Model note: Question text is clear; no OCR correction needed. The standard four access methods (sequential, direct, random, associative) are assumed. Some textbooks also list 'read-only' as a fifth category; the answer notes this but marks the standard four.*

## QUESTION A6 (10 marks)

*Topic: Data representation and number systems*

a) Answer: left 38 -> 0 0100110; right -28 -> 1 0011100; 38 - (-28) = 66 -> sign-magnitude 0 1000010

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left 38 -> 1's 00100110, 2's 00100110; right -28 -> 1's 11100011, 2's 11100100; 38 - (-28) = 66 -> 1's 01000010, 2's 01000010

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

## QUESTION A7 (10 marks)

*Topic: Input/output and interrupts*

*Source: CSC429 (14).pdf, page(s) 4*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) A maskable interrupt is a hardware interrupt that the CPU can temporarily ignore or postpone by setting/clearing an interrupt mask bit in its interrupt-enable register (or by the interrupt priority/mask logic). While masked, the interrupt request is held pending and is not serviced until it is unmasked; when unmasked and no higher-priority interrupt is being serviced, the CPU acknowledges it, saves the current context (PC/PSW), and branches to the corresponding interrupt handler. Maskable interrupts are used for ordinary I/O devices, in contrast to non-maskable interrupts (NMI), which cannot be disabled and are reserved for critical events.

(b) Defined priority (higher-priority interrupt preempts a lower-priority handler; equal/lower priority waits). Assuming the usual convention that the handlers are ordered A > B > C > D > E (A highest), the

© Hak Cipta Universiti Teknologi MARA 

execution sequence is: Main program → Interrupt A occurs, Handler A starts → Interrupt B occurs; B has lower priority than A, so B is held pending → Handler A completes → Handler B runs → Interrupt C occurs; C is lower than B, so C is held pending → Handler B completes → Handler C runs → Interrupt D occurs; D is lower than C, so D is held pending → Handler C completes → Handler D runs → Interrupt E occurs; E is lower than D, so E is held pending → Handler D completes → Handler E runs → return to Main program. Nested preemption would occur only if a later interrupt had higher priority than the handler currently running (e.g. if C arrived while B was running, C would preempt B). The exact nesting depends on the arrival times and priority order shown in Figure 2.

(c) Disable interrupt (interrupts disabled while a handler executes, i.e. non-nested/single-level handling). Once the first interrupt is accepted, further interrupts are disabled until that handler returns (interrupt enable restored on return). Sequence: Main program → Interrupt A accepted, interrupts disabled, Handler A runs to completion → interrupts re-enabled → pending Interrupt B accepted, Handler B runs to completion → Interrupt C accepted, Handler C runs to completion → Interrupt D accepted, Handler D runs to completion → Interrupt E accepted, Handler E runs to completion → return to Main program. No handler is preempted; each runs atomically in arrival order.

(d) Defined priority can finish Interrupt C earlier. Justification: under defined priority, a higher-priority interrupt (e.g. A or B) can preempt a lower-priority handler, so C is not forced to wait for the entire lower-priority handler to finish; higher-priority work is completed promptly and C is serviced as soon as the higher-priority handlers that preempt it complete. Under disable-interrupt handling, C must wait until the currently running handler (and any earlier pending ones) finishes completely, because interrupts are disabled during handler execution, so C's completion is delayed by the full duration of preceding handlers. Hence the priority scheme allows C to complete sooner (assuming C's priority permits it to preempt lower-priority handlers).

**Unofficial marking guide:**

*• 1 mark: defines maskable interrupt as one that can be disabled/ignored via an interrupt mask/enable bit*
*• 1 mark: states masked requests are held pending and serviced when unmasked (context save and branch to handler)*
*• 1 mark: correct overall ordering Main → A → B → C → D → E → Main under defined priority*
*• 1 mark: explains higher-priority interrupt preempts lower-priority handler*
*• 1 mark: explains lower/equal-priority interrupt is held pending until current handler completes*
*• 1 mark: correct nesting/preemption behaviour consistent with Figure 2 priorities*
*• 1 mark: correct sequence Main → A → B → C → D → E → Main with interrupts disabled during each handler*
*• 1 mark: states no preemption; each handler runs to completion before next is serviced*
*• 1 mark: identifies defined priority as finishing Interrupt C earlier*
*• 1 mark: justifies via preemption of lower-priority handlers vs. C waiting for full completion under disable-interrupt*
*Model confidence: medium.*
*Model note: Figure 2 is only available as extracted text; the actual arrival order and priority ordering of handlers A–E must be read from the figure. The answer assumes the common convention A highest → E lowest and that interrupts arrive in the order A, B, C, D, E as implied by the diagram; if the figure shows a different priority order or arrival timing, the nesting in (b) and the conclusion in (d) should be adjusted accordingly. This is an unofficial model answer, not a marking scheme.*

## QUESTION A8 (10 marks)

*Topic: Computer architecture and CPU organisation*

*Source: CSC429 (3).pdf, page(s) 3*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) Three types of system interconnection [6 marks]

1. Bus interconnection: A set of shared parallel conducting lines (address bus, data bus and control bus) that connects the CPU, main memory and I/O peripherals. Only one transfer can take place on the bus at a time, so it is simple and low-cost but becomes a bottleneck as more devices are added; performance is improved by using separate or hierarchical buses and wider/faster lines.

2. Point-to-point interconnection: A dedicated link (a set of wires or a serial lane) connects exactly one pair of components, e.g. CPU-to-memory or CPU-to-I/O controller. Because the link is not shared, it gives higher bandwidth and avoids arbitration contention; many such links can be combined into a switched fabric (as in PCI Express) so several pairs communicate simultaneously.

3. Switched/multistage interconnection network: Crossbar switches or multistage networks (e.g. omega network) route data between any input and any output through switching elements. This allows multiple simultaneous paths between processors, memories and peripherals, giving high throughput and

© Hak Cipta Universiti Teknologi MARA 

scalability at the cost of more hardware and switching delay. (Other acceptable answers: shared multiprocessor bus, mesh/torus network-on-chip, or direct memory access channel.)

(b) Two sub-components of the CPU and their functions [4 marks]

1. Control Unit (CU): Fetches and decodes instructions, generates the timing and control signals that direct the ALU, registers and memory/I/O, and sequences the fetch-decode-execute cycle. It thus coordinates and controls the operation of the whole computer.

2. Arithmetic Logic Unit (ALU): Performs arithmetic operations (add, subtract, multiply, divide) and logical operations (AND, OR, NOT, XOR, shift, compare) on data held in registers; it sets condition/status flags (zero, carry, sign, overflow) used for decision making.

(Also acceptable: registers such as PC, IR, MAR, MBR and the accumulator, with their functions of holding the address/instruction/data during execution.)

**Unofficial marking guide:**

- *1 mark: naming a valid type of system interconnection (bus, point-to-point, switched/multistage network)*
- *1 mark: explaining how the bus interconnection works and its limitation*
- *1 mark: explaining point-to-point interconnection and its advantage*
- *1 mark: explaining switched/multistage interconnection and its advantage*
- *2 marks: accurate explanation of the third interconnection type with its relative merit (total 6 for part a)*
- *1 mark: naming a CPU sub-component (Control Unit or ALU)*
- *1 mark: correct function of that sub-component*
- *1 mark: naming a second CPU sub-component*
- *1 mark: correct function of the second sub-component (total 4 for part b)*

*Model confidence: high.*

*Model note: Figure 1 is only available as extracted text; the answer assumes the standard bus-based von Neumann organisation shown (CPU, main memory, I/O, interconnection network). The question wording 'types of system interconnection' is interpreted as interconnection structures (bus, point-to-point, switched network); if the lecturer intended bus types (data/address/control), those are covered within the bus explanation. This is an unofficial model answer, not a marking scheme.*

## QUESTION A9 (10 marks)

*Topic: Computer architecture and CPU organisation*

*Source: CSC429 (8).pdf, page(s) 3*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a)(i) The size of a data block is determined by the width of the data bus. Here the data bus is 64 bits, so the data block size is 64 bits (8 bytes). Justification: the data bus carries the actual data between the CPU and memory/I/O; each transfer moves one data block equal to the data-bus width. The address bus only selects the location, so it does not determine block size.

(a)(ii) The address bus is used to carry the address of the memory location or I/O port that the CPU wants to access. It is unidirectional (from CPU to memory/I/O) and its width determines the number of addressable locations: with a 16-bit address bus, 2^16 = 65,536 (64K) distinct addresses can be selected.

(a)(iii) If a wider address bus is set, the number of addressable memory locations increases (address space grows as 2^n for an n-bit address bus). For example, moving from 16 to 32 bits increases the address space from 64K to 4G locations. This allows larger programs and more memory to be used, but it also requires wider address registers, wider internal address paths, and more address lines/pins, which increases cost and complexity. The data-block size is not changed by widening the address bus.

(b) Two types of bus timing are synchronous and asynchronous timing. Synchronous bus timing: all bus operations are referenced to a common clock; events occur on fixed clock edges/cycles. It is simple and fast for devices that can match the clock, but slow devices need wait states, and clock skew limits performance. Asynchronous bus timing: no common clock; a handshake protocol (e.g., request/acknowledge or master/slave strobes) coordinates the transfer. It accommodates devices of different speeds and is more flexible, but the handshake overhead makes it slower and more complex than synchronous timing.

**Unofficial marking guide:**

- *1 mark: data block size stated as 64 bits (or 8 bytes).*
- *1 mark: justification that data block size equals data-bus width / data bus carries one block per transfer.*
- *1 mark: address bus carries the address of memory location or I/O port to be accessed.*
- *1 mark: address bus is unidirectional from CPU and/or its width determines number of addressable locations (2^16*

© Hak Cipta Universiti Teknologi MARA 

*= 64K).*

*• 1 mark: wider address bus increases addressable memory/address space (2^n locations).*

*• 1 mark: consequence such as larger memory/programs possible, or increased pins/cost/complexity; data block size unchanged.*

*• 1 mark: synchronous bus timing described with common clock and fixed clock cycles/edges.*

*• 1 mark: synchronous advantage/limitation (simple/fast, but wait states or clock skew).*

*• 1 mark: asynchronous bus timing described with handshake/request-acknowledge and no common clock.*

*• 1 mark: asynchronous advantage/limitation (handles different speeds/flexible, but slower/more complex).*

*Model confidence: high.*

*Model note: Assumed 'data block' means the unit transferred per data-bus cycle, i.e. equal to data-bus width. No figure or table is required. The question is from an unofficial past paper; this is not an official marking scheme.*

## QUESTION A10 (10 marks)

*Topic: Input/output and interrupts*

*Source: CSC429 (12).pdf, page(s) 5*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) Two functions of an I/O module (4 marks): (1) Interface/translation between the CPU and peripheral devices — the I/O module acts as an intermediary that converts the CPU's internal data format and control signals into the timing, voltage and protocol required by the external device, and vice versa. It hides device-specific details from the processor so the CPU can use a uniform set of I/O instructions/addresses. (2) Data buffering and transfer control (synchronisation) — the module temporarily holds data in registers (e.g. data-in/data-out buffers) to compensate for the speed mismatch between fast CPU/memory and slow peripherals, and it manages the transfer (e.g. status/control registers, handshaking, generating interrupt requests or DMA requests) so data is moved only when the device is ready. Other acceptable functions: error detection (parity), device addressing/selection, and control/status reporting.

(b) Two categories of external I/O devices with examples (4 marks): (1) Human-readable / user-interface devices — devices that communicate with the user, e.g. keyboard, mouse, monitor (VDU), printer. (2) Machine-readable devices — devices that communicate with other equipment/machines, e.g. disk drive, magnetic tape drive, sensor/actuator, network interface card. (The third standard category is communication devices, e.g. modem/NIC.)

(c) I/O techniques (2 marks): Requires active CPU intervention — programmed I/O (polling), in which the CPU repeatedly checks the device status and performs the transfer itself. Does not require active CPU intervention — DMA (direct memory access), in which a DMA controller moves data between memory and the device while the CPU continues other work (interrupt-driven I/O is also acceptable as it frees the CPU from polling, though the CPU still services the interrupt).

**Unofficial marking guide:**

*• 1 mark: correct identification of the first function of an I/O module (e.g. interfacing/format and signal translation between CPU and device)*

*• 1 mark: discussion/explanation of that first function*

*• 1 mark: correct identification of the second function (e.g. buffering/synchronisation, control and status, error detection, addressing)*

*• 1 mark: discussion/explanation of that second function*

*• 1 mark: correct naming of one device category (e.g. human-readable)*

*• 1 mark: appropriate example for that category (e.g. keyboard/monitor/printer)*

*• 1 mark: correct naming of a second device category (e.g. machine-readable)*

*• 1 mark: appropriate example for that category (e.g. disk drive/sensor)*

*• 1 mark: names an I/O technique requiring active CPU intervention (programmed I/O/polling)*

*• 1 mark: names an I/O technique not requiring active CPU intervention (DMA or interrupt-driven I/O)*

*Model confidence: high.*

*Model note: Question text has minor OCR/grammar issues ('categories' repeated); interpreted conservatively. Part (c) accepts interrupt-driven I/O as the non-CPU-intervention technique, though DMA is the strongest answer since the CPU is not actively involved in the transfer itself.*

© Hak Cipta Universiti Teknologi MARA 

# PART B — MARKING GUIDE

## QUESTION B1 (20 marks)

*Topic: Assembly-language programming*

*Source: CSC429 (20).pdf, page(s) 5*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

Assumption on the equation (OCR correction): Figure 1's extracted text reads z=A2-B*C+jl_ D+5, which is garbled. The most conservative reading consistent with the instruction set (which has ADD, SUB, MUL, DIV but no square-root or exponent instruction) is z = (A² − B·C) / (D + 5). The jl_ is taken as a division bar /, A2 as A² (computed as A·A), and D+5 as the denominator. All three programs below compute this expression and leave the result in z. (If the intended equation differs, the same structure applies; see notes.)

Three-address program (each instruction names two sources and one destination; A, B, C, D are fixed registers, x, y, z are temporaries):

```
MUL (A, A, x)     ; x = A * A
MUL (B, C, y)     ; y = B * C
SUB (x, y, x)     ; x = A^2 - B*C   (numerator)
ADD (D, 5, y)     ; y = D + 5       (denominator)
DIV (x, y, z)     ; z = (A^2 - B*C) / (D + 5)
```

The three-address form is the most compact: one instruction per operation, no explicit data movement, because operands and result are all named in the instruction.

One-address program (an implicit accumulator; LOAD/STOR move between memory/registers and the accumulator, ADD p etc. do ACC ← ACC op p):

```
LOAD A          ; ACC = A
MUL A           ; ACC = A * A
STOR x          ; x = A^2
LOAD B          ; ACC = B
MUL C           ; ACC = B * C
STOR y          ; y = B*C
LOAD x          ; ACC = A^2
SUB y           ; ACC = A^2 - B*C
STOR x          ; x = numerator
LOAD D          ; ACC = D
ADD 5           ; ACC = D + 5
STOR y          ; y = denominator
LOAD x          ; ACC = numerator
DIV y           ; ACC = numerator / denominator
STOR z          ; z = final answer
```

The one-address form needs explicit LOAD/STOR because only one operand (the accumulator) is implicit; intermediate results must be spilled to x and y.

Zero-address program (a stack machine; PUSH p pushes operand p, POP p pops the top into p, and ADD/SUB/MUL/DIV pop two operands and push the result):

```
PUSH A          ; stack: A
PUSH A          ; stack: A, A
MUL             ; stack: A*A
PUSH B          ; stack: A*A, B
PUSH C          ; stack: A*A, B, C
MUL             ; stack: A*A, B*C
SUB             ; stack: A*A - B*C
PUSH D          ; stack: num, D
PUSH 5          ; stack: num, D, 5
ADD             ; stack: num, D+5
DIV             ; stack: num / (D+5)
POP z           ; z = final answer
```

The zero-address form uses the stack for all operands and results; note the operand order (push

© Hak Cipta Universiti Teknologi MARA 

numerator first, then denominator) so that DIV computes numerator ÷ denominator.

Summary: all three programs compute z = (A² − B·C)/(D + 5) and store the final result in z; they differ only in how operands are addressed (explicitly named, via an accumulator, or via a stack).

**Unofficial marking guide:**

*• 2 marks: correct interpretation/statement of the equation to be computed (A² − B·C)/(D + 5), including A² as A·A and D+5 as denominator*

*• 6 marks: correct three-address program — 1 mark each for computing A·A, B·C, the subtraction (numerator), D+5 (denominator), the division, and storing the result in z*

*• 6 marks: correct one-address program — 1 mark each for correct use of LOAD/STOR to form A², B·C, numerator, denominator, division, and final STOR z (accumulator semantics correct)*

*• 6 marks: correct zero-address program — 1 mark each for correct PUSH sequence for A², correct PUSH/MUL for B·C, correct SUB for numerator, correct PUSH/ADD for D+5, correct DIV with operands in the right order, and POP z for the final result*

*Model confidence: medium.*

*Model note: The equation in Figure 1 is OCR-garbled ('z=A2-B*C+jl_ D+5'). I conservatively read it as z = (A² − B·C)/(D + 5), since the instruction set has no exponent or square-root operation and A² must be computed as A·A. If the actual figure shows a different expression (e.g. a different denominator or an added term), the same three program structures apply but the exact instruction sequences and mark allocation per step would change. The register names A, B, C, D are treated as fixed registers and x, y, z as temporaries, with z holding the final answer, per the question. This is an unofficial model answer, not a marking scheme.*

*END OF MARKING SCHEME*

© Hak Cipta Universiti Teknologi MARA 